

Computer Security Exam

Professors M. Carminati & S. Zanero

Milan, 22/07/2019

Last (family) Name _____

First (given) Name _____

Matricola or Codice Persona _____

Have you done any challenges/homework, even partially?	☐ Yes	☐ No
Professor:	☐ Carminati	☐ Zanero
Course:	☐ Milan	☐ Como

Instructions

- The exam is composed of 15 pages. Check that you have all of them
- Just as a cross check, tell us whether you have completed the homework, by putting an "X" mark appropriately.
- The exam is "closed books". Please put away in a non-suspicious place (i.e. not below the desk) any notebook, or similar. You will be expelled if, at any time, if you do not follow this rule.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Shut down and store electronic devices. They will be subject to inspection if found and you may be expelled if you are found using one.
- Please answer within the allowed space. Schemes are good, short answers are recommended.
- You can write in pen or pencil, any color, but avoid writing in red.
- No extra paper is allowed.
- The answers should be written exclusively in the space provided below the questions.
- **Always motivate your answer.**

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

SOLUTION

Answer provided in this solution **MUST BE CONSIDERED ONLY AS A HINT** for the correct answer, and they are not necessarily complete.

Question 1 (10 points)

Consider the C program below.

```
01 #include <stdio.h>
02 #include <stdlib.h>
03 #include <string.h>
04
05 int door(char *user) {
06     struct {
07         char user[4];
08         char pass[16];
09         char realpass[16];
10         char ring[8]="FRIENDS";
11         int fd;
12     } s;
13
14     snprintf(s.user, 20, user);
15     printf("Welcome user: %s \n", s.user);
16     s.fd = open("pass.txt", O_RDONLY);
17     read(s.fd, s.realpass, 15); /* load password into memory */
18     s.realpass[15] = '\0';
19     fclose(s.fd)
20
21     printf("The Doors of Durin, Lord of Moria. Speak, friend, and enter.");
22
23     gets(s.pass);
24     if(!strncmp(s.pass, "P4SS", 4) && !strncmp(s.pass, s.realpass, 12))
25     {
26         printf("Mellon: %s", s.ring);
27         return 1; /* Access OK */
28     } else {
29         printf("You shall not pass");
30         abort(); /* The program exits and the function does not return */
31     }
32 }
33
34 int main(int argc, char** argv) {
35     door(argv[1]);
36 }
```

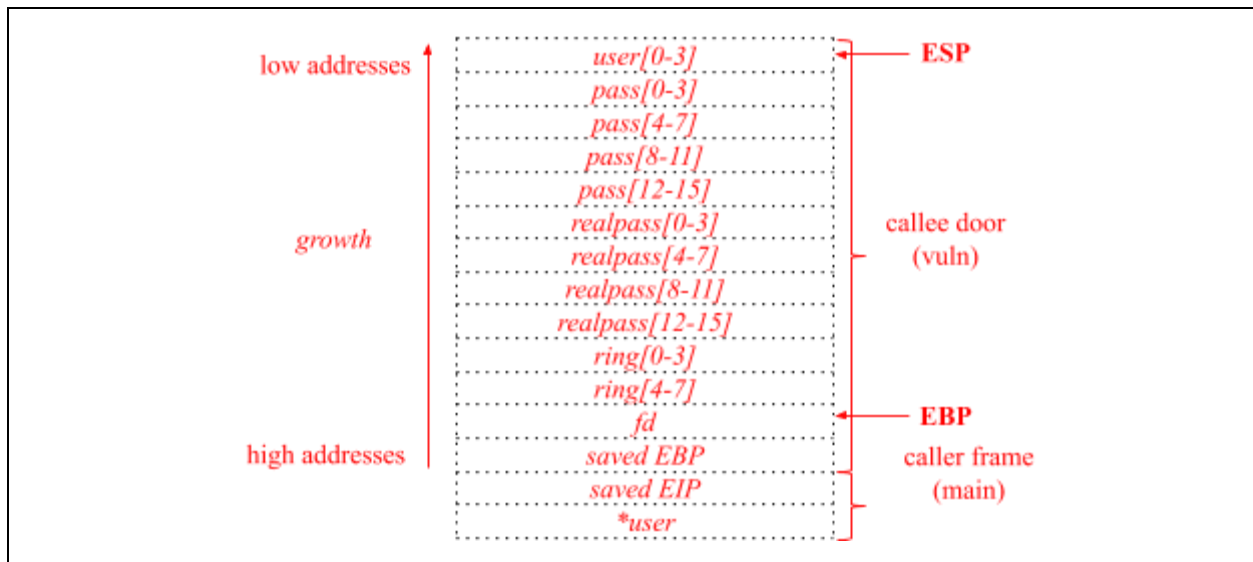
1. [1 point] The program is affected by typical buffer overflow and format string vulnerabilities. Complete the following table.

Vulnerability	Line	Motivation
Buffer Overflow	Line 23 Line 14	<i>the gets reads an user-supplied string of arbitrary length and copies it in a stack buffer</i>
Format String	Line 14	

2. [1 point] Assuming that the program is compiled and run for the usual IA-32 architecture (32-bits), with the usual cdecl calling convention, draw the stack layout just before the execution of line 13, i.e., at the beginning of the function `door()`, showing:

- Direction of growth and high-low addresses;
- The name of each allocated variable;
- The boundaries of the function stack frames (`main` and `door`)

Show also the content of the caller frame (you can ignore the environment variables: just focus on what matters for the exploitation of typical memory corruption vulnerabilities).

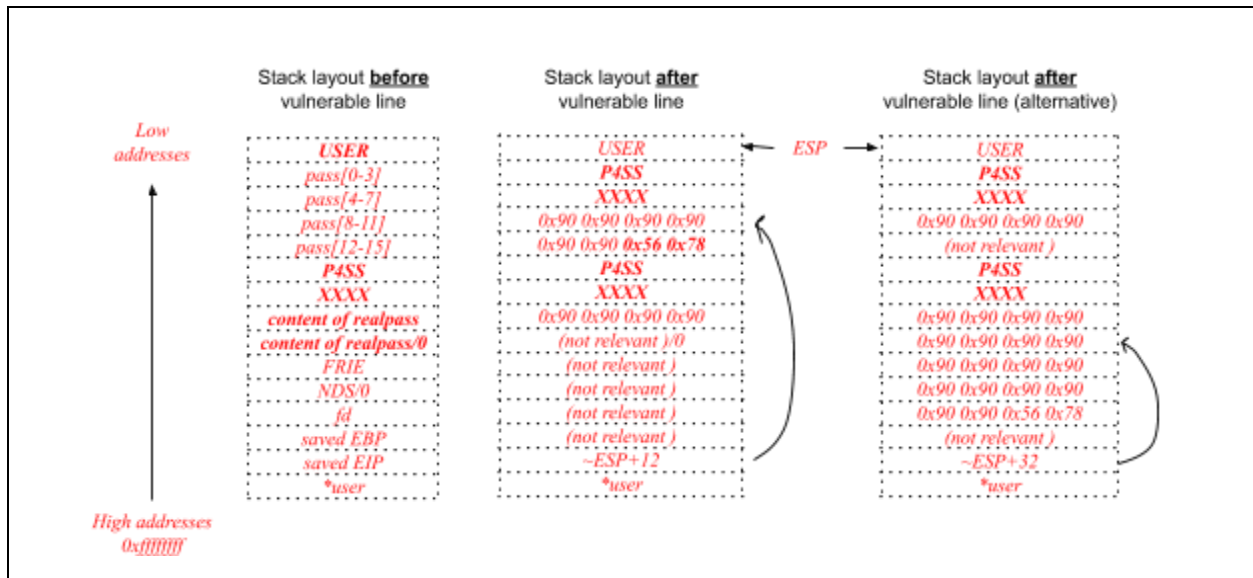


4. Focus only on the **Buffer Overflow vulnerability** you identified.

4.1 [2 points] Assume that the program is compiled and run with no mitigation against exploitation of memory corruption vulnerabilities (**no canary, executable stack**, environment **with no ASLR** active).

Write an exploit for the buffer overflow vulnerability identified to bypass the authentication mechanism. Your exploit **must** execute the following simple shellcode composed only by 2 instructions (2 bytes) that bypasses the authentication mechanism: `0x56 0x78`.

Write the exploit clearly, detail all the steps and assumptions you need for a successful exploitation, and draw the stack layout right before and after the execution of the detected vulnerable line during the program exploitation.



4.2 [2 points] Now assume that the stack is not executable and that the program is compiled with correctly implemented stack canary (ASLR is not active). Is the exploit you just wrote still working without modifications? If not, explain why and propose an alternative solution to exploit the vulnerability and bypass the authentication mechanism (your alternative solution does not necessarily require to execute the shellcode).

No, because ...[SEE SLIDES]...

Alternative solution:

We exploit the same buffer overflow vulnerability at line 23 to overwrite pass and realpass (stopping the overflow before the correctly implemented canary) in order to satisfy the following condition “if(!strcmp(s.pass, "P4SS", 4) && !strcmp(s.pass, s.realpass, 12))” and bypass the authentication.



5. This time focus only on the format string vulnerability you identified.

5.1 [1 point] Write an exploit for the format string vulnerability to execute a piece of shellcode saved in an environment variable that discloses the content of the file “pass.txt”. Assume that ASLR is not active and, from a previous execution of your program, you know that:

- The address of the environment variable containing your shellcode (i.e., **what to write**) is 0x793b96e6
 - 0x793b(hex) = 31035(dec), 0x96e6(hex) = 38630(dec)
- The address of the saved return address (i.e., the address **where we want to write**) is 0x43063ab8
- The **displacement on the stack** of your format string is equal to 4

Write the exploit clearly, detailing all the components of the format string, and detailing all the steps that lead to a successful exploitation.

The command line argument is directly fed as a format string to snprintf, and the format string itself is on the stack with a given displacement: we just need to construct a standard format string exploit.

- We need to write 0x793b96e6 to 0x43063ab8 (<tgt>)
- 0x793b < 0x96e6 -> we write 0x793b first

The value of the command line argument will have the form

<tgt+2><tgt>%<N1>c%<pos>\$hn%<N2>c%<pos+1>\$hn

where

```
<tgt+2> = 0x43063aba  
<tgt> = 0x43063ab8  
<pos> = 4  
<N1> = dec(0x793b) - 8 (bytes already written) = 31035 - 8 = 31027  
<N2> = dec(0x4543) - 16961 (bytes already written) = 38630 - 31035 = 7595
```

thus the final string is

```
\xba\x3a\x06\x43\xb8\x3a\x06\x43%31027c%4$hn%7595c%5$hn
```

5.2 [1 point] If only address space layout randomization (ASLR) is active, is the exploit you just wrote still working *without modifications*? Why? If not, explain why and propose an alternative solution to exploit the vulnerability.

No, ... [SEE SLIDES] ...

5.3 [1 point] If the stack is only made non executable (i.e., NX, DEP, or W^X), is the exploit you just wrote still working *without modifications*? Why?

No, the exploit executes shellcode from the stack and therefore ...

5.4 [1 point] If the program is compiled only with correctly implemented stack canary, is the exploit you just wrote still working *without modifications*? If not, propose an alternative solution to exploit the program.

Yes, ... [SEE SLIDES] ...

Question 2 (11 points)



Consider **STRANGER LISTS**, a web application that manages a shared TODO list. The application allows registered users to create a TODO list and to share individual list items with other users. Each list item is composed of a text and a due date, and can be shared with multiple users. Shared items appear as part of the TODO list.

Consider the following code snippets:

Snippet 1 – Display TODO list (<https://todo.strangerlists.com/view>)

```
00 function view_list(request):
01     user = check_session(request)
02     if user is None or request.method != "GET":
03         abort(403)
04     q = "select * from todo where owner = " + user +
05         " union select * from todo join share on todo.id = share.todo_id " +
06         " where share.username = " + user + " order by due_date desc"
07     result = db.query(q)
08     if result is None:
09         abort(403)
10     page = "<h1>TODO List</h1>"
11     page += "<table><tr><td>Due date</td><td>Text</td><td>Owner</td>"
12     foreach row in result:
13         page += "<li>" . htmlentities(row['id']) . "</li>"
14         page += "<li>" . htmlentities(row['due_date']) . "</li>"
15         page += "<li>" . htmlentities(row['text']) . "</li>"
16         page += "<li>" . htmlentities(row['owner']) . "</li>"
17     return render(page)
```

Snippet 2 – Create TODO list item (<https://todo.strangerlists.com/create>)

```
18 function create_item(request):
19     user = check_session(request)
```

```
20     if user is None or request.method != "POST":
21         abort(403)
22     q = "insert into todo (owner, due_date, text) values ( ' " + user + " ', ' "
23         + request.due_date + " ', ' " + request.text + " ')"
24     if db.query(q) == True:
25         return "<p>Item added!</p>"
26     return "<p>Error adding item</p>"
```

Snippet 3 – Share a TODO list item (<https://todo.strangerlists.com/share>)

```
27 function share_item(request):
28     user = check_session(request)
29     if user is None or request.method != "POST":
30         abort(403)
31     q = "insert into share (todo_id, username)
32         values ( ' " + request.todo_id + " ', ' " + request.username + " ')"
33     if db.query(q) == True:
34         return "<p>Item shared!</p>"
35     return "<p>Error sharing item</p>"
```

Assume the following:

- the function `check_session()` securely manages the users' sessions
- the function `htmlentities()` converts special characters such as `<`, `>`, `"`, and `'` to their equivalent HTML entities (i.e., `<`, `>`, `"` and `'` respectively)
- user registration and login are securely implemented

The web application uses the following database tables:

```
CREATE TABLE users (
    user_id INTEGER PRIMARY KEY AUTOINCREMENT,
    username VARCHAR(100),
    password_hash VARCHAR(256)
);
CREATE TABLE todo (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    owner VARCHAR(100),
    due_date DATETIME,
    text TEXT
);
CREATE TABLE share (
    todo_id INTEGER,
    username VARCHAR(100),
    PRIMARY KEY(todo_id, username)
);
```


Clearly, the web application is affected by one or more SQL injection vulnerabilities.

1. [1 point] Specify the line(s) of code that introduce the vulnerability(ies) and explain briefly which conditions must hold on a website in order to expose a SQL injection vulnerability.

S2 L22-23
S3 L31-32

Conditions: ... [SEE SLIDES] ...

2. [2 points] Write an exploit to insert a note owned by the user "alexei.smirnoff" and shared with "jim.hopper". Please describe all the steps and assumptions you need to make for a successful exploitation. Assume you are logged in as the user 'thirteen'.

It's an injection in a database insert. Exploit: write a TODO item with the following text: text', ('alexei.smirnoff', '2019-10-10 00:00:00', 'USSR

To share the note with 'jim.hopper', we need to get the ID of the new item.

We can exploit the previous vulnerability to retrieve the ID of the inserted item

text', ('thirteen', NOW(), ' (select id from todo where owner = alexei.smirnoff and due_date=2019-10-10 00:00:00 and text LIKE 'USSR

Or just notice that the IDs are sequential, thus I can see the ID of the note I just added owned by myself with a call to view_list then add 1 and get the ID of the note I injected.

Assuming that the ID is 1001, just need to visit share_item with parameters 1001 and 'jim.hopper'.

3. [1 point] Please describe the simplest procedure to mitigate, or avoid completely, the vulnerability you just exploited, assuming you can change the source code of the web application.

Use prepared statements (alternative answer, albeit less correct, especially in light of question 5: escape or filter all sensitive characters such as ')

4. [1 point] Assume that you can't change the code of the web application. Can you mitigate, or avoid, the exploitation of this vulnerability (to achieve the effect mentioned in question 2.2) at the *database administration* level? Why?

No (unless the DBMS allows to restrict the number of tuples inserted by a single query somehow).

5. [2 points] Assume you have the capabilities to log and decode all the network traffic *between the web application and the database* (i.e., you have a log of all the queries and their responses). Can you *detect* when the vulnerability is being exploited to obtain the specific goal of question 2.2? Why? Is there a way for an attacker to exploit the vulnerability without it being detected? Why?

I can check whether all the queries consists of a single or multiple tuples and allow only inserts with a single tuple. To achieve the above goal, an attacker needs to exploit a SQL injection to insert a second tuple in the original INSERT, thus it is necessarily detected. More in general: in this setting, one can assume that the structure of the queries performed by the application don't change often and raise an alarm every time the structure of an executed queries changed (SELECTs with multiple UNIONS, INSERTs with multiple tuples, subqueries, ...).

No,

6. [1 point] Assume you are configuring a *reverse web proxy* in front of the web application, i.e., you can modify and inspect all the application-layer network traffic (HTTP requests and responses) to and from the web application. Suggest what is the simplest procedure you can implement to prevent, or mitigate, the exploitation of the vulnerability.

We can filter all requests containing characters that have a meaning in a SQL context, e.g., ', ", ...

7. [1 point] Assume that the SQL injection has been completely mitigated or resolved. Write an exploit for a (different) vulnerability that results in an existing note (belonging to an arbitrary user, who is not you) with ID 1000 to be shared with 'm.carminati'. Describe the root cause of this vulnerability and explain the simplest procedure to remediate it.

It is a logical vulnerability. The function `share_item` doesn't check whether the note is owned by the user who is logged in. Procedure to remediate: add a check that the owner of the note is the logged in user.

8. [2 points] Consider the class of vulnerabilities known as CSRF (cross site request forgery). Clearly, the web application is vulnerable to CSRF.

8.1. Explain briefly which conditions must hold on a website in order to expose a CSRF vulnerability.

The website must allow an authenticated user to perform a state-changing action via a request (be it a GET or a POST request), which is authenticated using cookies (ambient credentials, sent automatically by the browser for every request).

Furthermore, the website does not implement any technique to validate whether the request is initiated from the same website or from a different website (i.e., whether the request is intentional or not), such as by using CSRF tokens or validating the Origin header.

If the conditions hold, a malicious actor can lure the user to go to an attacker-controlled website which will trigger a GET or a POST request to the vulnerable endpoint from the user's browser and on behalf of the authenticated user.

8.2. To remediate to this vulnerability, the developers implemented the following technique:

1. When a page that allows the user to perform a state-changing action (e.g., a page which contains a form) is visited, the server generates a token. A token is a random alphanumeric string of 100 characters.
2. The token is sent in the response as a cookie (i.e., `Set-Cookie: csrf_check=<the token>`)
3. When the form is submitted, the backend checks whether there is a valid `csrf_check` among the cookies sent by the client

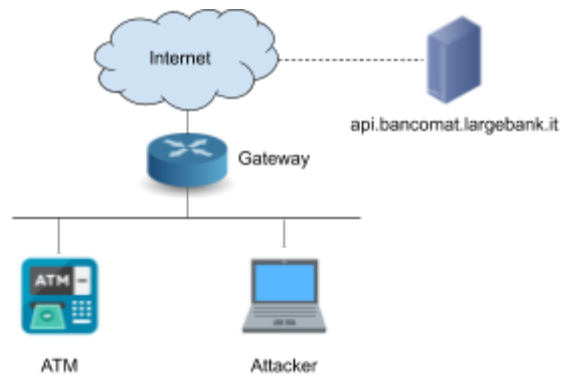
Is this technique effective to mitigate the CSRF vulnerability? If you think it's effective, please explain why. If you think it's not effective, please explain why it is not effective, describe a bypass and suggest a correct solution.

It is not effective. Indeed, an attacker just needs to send two requests: one to trigger the Set-Cookie and another to perform the state changing action. Both requests can easily be caused from the same attacker-controlled page (e.g., insert an `<img src>` tag to cause a GET to the page containing the form and an auto-submit form to submit the form). The correct solution is to force the client to read the response before being allowed to submit data. This can be easily achieved by placing the CSRF token in a hidden form field instead of a cookie.

Question 3: (7 points)

A large bank is considering to deploy a set of ATMs (for Italian speaking students: “bancomat”) in various *Starcourt Malls*, geographically distributed throughout the city of Hawkins. ATMs are connected to the Internet through the shopping mall’s LAN. To exchange data with the bank servers, ATMs use a web service exposed via HTTP and accessible at a known URL (e.g., <http://api.bancomat.largebank.it>).

1. [2 points] Assume that, at a certain shopping mall, the ATM is connected to the same (Internet-connected) LAN of the guest network (see the following picture).



You are an attacker, connected to the guest network, and willing to tamper with the data exchanged between the ATM and the bank to change the amount of money charged for a certain transaction. Describe an attack to carry out this goal, detailing all the steps and assumptions you need in this scenario.

As the attacker and the ATM are connected to the same LAN, the attacker can perform ARP cache poisoning. The attacker sends out gratuitous ARP answers for the gateway IP with its own MAC address (directed to the ATM IP) and gratuitous ARP answers for the ATM IP with its own MAC address (directed to the gateway). This way, the attacker is a man in the middle between the ATM and the network gateway and can intercept and modify all the traffic between those endpoints. At this point, the attacker just needs to change the amount of the transactions sent through the HTTP request from the ATM to the bank.

2. [2 points] Assume you are managing the shopping mall's network (i.e., you control all the networking equipment, such as the network gateway and switches, including their configuration and code).

2.1. Is there any way to detect whether the attack you just described is going on in this network?

At the gateway I can detect whether the MAC address for a given IP address is changing (particularly the MAC address for the ATM IP address) and raise an alarm or, more in general, I can detect whether there are multiple responses for an ARP request and raise an alarm.

2.2. Is there any way to prevent this attack (without changing the network layout)?

Various answers are possible. The general idea is ARP inspection, e.g., as implemented in CISCO switches: <https://www.cisco.com/c/en/us/td/docs/switches/lan/catalyst4500/12-2/25ew/configuration/guide/conf/dynarp.html>. A more simplistic (valid) answer is to use the detection procedure of point 2.1. and consider that for some devices (ATM and gateway) the MAC address does not change or changes rarely and always consider the first received MAC address.

To mitigate this issue, the network administrators separated the LAN into two subnets (i.e., two broadcast domains): a network for the ATM and a network for the guest access (see figure below).



3. [1 point] A network gateway connects and provides Internet access to both networks. The network gateway does not implement any firewall functionalities (i.e., the attacker in the guest network can connect to the ATM IP and vice-versa). Is your attack from point 1. still working in this scenario? Why?

Answer: No

Broadcasting...

4. [2 points] Assume the same scenario of Question 3.1 and assume you are hired from the bank's security team to improve the security of this scenario (assume you can't change the network setup and layout, but only the implementation of the software and protocols of the ATM and of the bank web services). Detail what technique you would implement to avoid a man in the middle attack, explaining why they are effective in this scenario.

Answer: Use TLS (bonus: with client authentication) / use a VPN

Question 4 (5 points)

1. [2 points] Consider the security model of the Android mobile platform. Please briefly describe how the app sandboxing and the permission-based authorization model work.

Answer (see slides):

- *Sandboxing: each app is assigned to a distinct Linux user*
- *Permissions: each app declares a set of permissions in the manifest file, permissions need to be granted at app installation (Android < 6) or at first use (Android >= 6).*

2. [1 points] Consider now mobile malware developed for the Android platform. Can an antivirus for Android be as effective as an antivirus developed for a legacy operating system (e.g., Windows, or Linux) in the task of detecting a malware already running in the system? Why?

Answer: No because of sandboxing (see slides)

3. [2 points] Suppose you are about to install a game for your Android phone from the official app store. While installing the app, you observe that the application, in the manifest, requests (among others) the following permissions:

`<uses-permission="android.permission.SEND_SMS">` (send SMS messages)

`<uses-permission="android.permission.READ_CONTACTS">` (access address book)

You suspect that the app is indeed a piece of malware. Focusing only on the requested permissions, please describe at least two actions that are likely to be implemented by the malware (focus on the actions that represent the most likely threat).

Answer:

- *send sms to premium numbers (SEND_SMS)*
- *exfiltrate contacts through SMS (READ_CONTACTS + SEND_SMS)*
- *propagate by sending SMS to the contacts with a link to download the malware (READ_CONTACTS + SEND_SMS)*